

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

A.R.Yuvansri

192421029

1. Real-Time Face Recognition System (Edge Deployment)

Objective

Design a lightweight face recognition system for low-power edge devices such as Raspberry Pi. The system should process live video with low latency, handle lighting variations, use limited CPU and memory, and recognize people from a predefined database.

Pipeline

Image Acquisition → Preprocessing → Face Detection → Face Alignment → Feature Extraction → Feature Matching/Recognition → Result Display

1. Image Acquisition

Use OpenCV VideoCapture to read frames from a webcam or camera module. Capture at a moderate resolution such as 640×480 and process only the required frames to reduce CPU usage.

2. Preprocessing

Convert frames to grayscale when appropriate, resize them, and apply histogram equalization or CLAHE to improve robustness under varying illumination. Mild Gaussian filtering can reduce sensor noise.

3. Face Detection

Use a lightweight OpenCV Haar Cascade or a compact DNN face detector. Haar Cascades require less computation, while a lightweight DNN generally provides better robustness to pose and lighting changes.

4. Face Alignment

Crop detected faces and normalize their size. If facial landmarks are available, align the eyes and other key points to a standard orientation. This reduces variations caused by head pose.

5. Feature Extraction

Use a compact face descriptor such as an LBP-based descriptor for a simple CPU-friendly implementation, or a lightweight face-embedding model when higher recognition accuracy is required.

6. Recognition

Compare the extracted feature/embedding with the predefined face database using Euclidean distance or cosine similarity. Assign the identity when the similarity exceeds a selected threshold; otherwise label the face as Unknown.

Justification

Real-time performance is achieved through low-resolution input, frame skipping, lightweight detectors, ROI-based processing, and avoiding unnecessary color conversions. CLAHE improves illumination robustness. A compact feature representation reduces memory requirements, while threshold-based matching allows recognition against a predefined dataset.

2. Automated Video Surveillance with Alert System

Objective

Develop an OpenCV surveillance system that detects motion and unusual activity, produces real-time alerts, accepts standard CCTV feeds, and minimizes false positives in dynamic environments.

Pipeline

CCTV/Video Input → Frame Preprocessing → Background Modeling → Motion Detection → Noise Removal → Object/Feature Analysis → Temporal Activity Analysis → Alert Decision → Alert Output

1. Video Acquisition

Use OpenCV VideoCapture to receive CCTV streams or recorded video. Resize frames and process at a controlled frame rate to maintain real-time performance.

2. Preprocessing

Convert frames to grayscale and apply Gaussian or median filtering to suppress sensor noise. Morphological opening and closing can remove small isolated regions and fill gaps in motion blobs.

3. Motion Detection

Use background subtraction such as MOG2 or KNN to obtain a foreground mask. These methods adapt to gradual background changes better than simple frame differencing.

4. Feature/Object Analysis

Find contours and connected components in the foreground mask. Use bounding-box size, aspect ratio, centroid, and movement direction to reject tiny noise regions and characterize moving objects.

5. Unusual Activity Analysis

Track objects across consecutive frames and analyze speed, direction, dwell time, entry into restricted regions, sudden movement, or abandoned-object behavior. An alert is generated only when activity satisfies defined temporal and spatial rules.

6. Alert Generation

Display an on-screen warning and optionally save a snapshot or short video clip. Alerts can also be forwarded to an external notification service when integrated with the deployment environment.

Justification

Adaptive background subtraction handles gradual changes in dynamic scenes. Filtering and morphological operations reduce noise-related false positives. Minimum contour-area thresholds reject small disturbances, while temporal confirmation prevents a single noisy frame from triggering an alert. Frame resizing and selective processing maintain real-time operation.

6. Real-Time Object Detection on Mobile Device

Objective

Design an efficient camera-based object detection system for mobile devices with limited CPU and memory.

Pipeline

Camera Frame → Resize/Preprocess → Lightweight Object Detector → Non-Maximum Suppression → Bounding Boxes + Class Labels → Display

1. Image Acquisition

Capture frames using the mobile camera through an OpenCV-compatible camera interface. Use a reduced input resolution suitable for the detector.

2. Preprocessing

Resize the frame to the model input size and normalize pixel values. Avoid expensive operations that are not required by the detection model.

3. Detection

Use a lightweight detector such as a compact YOLO/SSD model with OpenCV DNN. The detector produces bounding boxes, class probabilities, and confidence scores.

4. Postprocessing

Discard detections below a confidence threshold and apply Non-Maximum Suppression (NMS) to remove overlapping duplicate boxes.

5. Display

Draw bounding boxes and class names on the current frame and display the result with minimal graphical overhead.

Justification

Efficiency is obtained by using a lightweight model, smaller input images, hardware acceleration where available, frame skipping when necessary, and processing only the latest camera frame. NMS and confidence filtering reduce unnecessary display and processing work.

10. Vehicle Detection in Parking System

Objective

Detect vehicles entering and exiting a parking area, track their movement, update available slots, and avoid double counting.

Pipeline

Camera/CCTV → Preprocessing → Vehicle Detection → Multi-Object Tracking → Entry/Exit Line or ROI Analysis → Slot State Update → Parking Availability Display

1. Video Acquisition and Preprocessing

Read the parking-camera stream using OpenCV. Resize frames and apply light denoising when required.

2. Vehicle Detection

Detect cars using a lightweight object detector. For a fixed parking camera, predefined parking-slot ROIs can also be used to determine whether each slot is occupied.

3. Tracking

Assign a unique temporary ID to each detected vehicle and track its centroid across frames. Simple centroid tracking can be used for controlled scenes; a stronger tracker can be used when vehicles frequently overlap.

4. Entry/Exit Counting

Define virtual entry and exit lines. Count a vehicle only when its tracked centroid crosses a line in the expected direction. The same track ID should not be counted repeatedly.

5. Slot Update

For slot-based parking, calculate vehicle overlap with each slot ROI or use occupancy detection. Update the database/state only after the vehicle remains in the slot for several frames, preventing flicker.

Justification

Tracking IDs prevent repeated counting across consecutive frames. Direction-based line crossing distinguishes entry from exit. Temporal confirmation and ROI-based occupancy reduce errors caused by detection noise, brief occlusion, or vehicles near slot boundaries.

11. Gesture Recognition System

Objective

Build a real-time webcam system that captures hand gestures, extracts useful features, classifies simple gestures, and responds with minimal delay.

Pipeline

Webcam → Preprocessing → Hand/Region Detection → Segmentation → Feature Extraction → Classification → Gesture Action

1. Image Acquisition

Capture webcam frames with OpenCV. Resize frames to a manageable resolution while retaining sufficient hand detail.

2. Preprocessing and Hand Segmentation

Convert the image to HSV or another suitable color space and use thresholding when the environment permits. Morphological filtering removes small noise. In a more robust implementation, a hand detector can first define the hand ROI.

3. Feature Extraction

Extract contour-based features such as contour area, perimeter, convex hull, convexity defects, bounding-box dimensions, aspect ratio, and centroid. These features are useful for simple gestures such as open palm, fist, or pointing.

4. Classification

For a small set of simple gestures, use rule-based classification with geometric thresholds. For more gestures, train a lightweight classifier such as KNN, SVM, or a small decision tree using the extracted features.

5. Real-Time Response

Map each recognized gesture to an action such as displaying a command, changing an application state, or controlling a simple interface. Use temporal smoothing so that the same gesture must be recognized consistently for several frames.

Justification

Contour and geometric features are computationally inexpensive and suitable for real-time processing. ROI processing reduces the number of pixels analyzed. Temporal smoothing prevents accidental responses caused by single-frame misclassification, while lightweight classifiers keep CPU and memory usage low.

Overall Design Considerations

Across all five systems, real-time performance can be improved using reduced frame resolution, ROI-based processing, frame skipping, lightweight OpenCV algorithms, efficient data structures, and hardware acceleration where supported. Noise suppression should be applied only when necessary because excessive filtering increases latency and can remove useful features. Thresholds such as confidence, contour area, similarity, and temporal persistence should be calibrated using representative deployment conditions.